Menduner 技术文稿

苏州城市活动情报平台 · 系统设计与工程说明

面向项目组内部同学

线上地址：http://47.116.133.230

版本 1.6 · 2026 年 10 月

阅读指南

这份文稿是给要动代码的同学看的，不是给老师看的汇报材料。目标是：读完你能知道系统由哪些部分组成、数据从哪来到哪去、哪条规则绝对不能改、以及一个新来源要怎么接进来。

按目的查

你想做	看第几节
搞懂系统整体怎么跑	1 架构总览
改前端页面或加组件	6 前端结构
加一个新的采集来源	3 采集管线 · 4 新增来源的完整步骤
改核验规则 / 加字段	5 核验规则（改动前必读）
查接口字段名	7 API 契约
本地跑起来	9 开发环境
上线部署	9 部署
知道哪些坑已经踩过	11 已知局限与陷阱

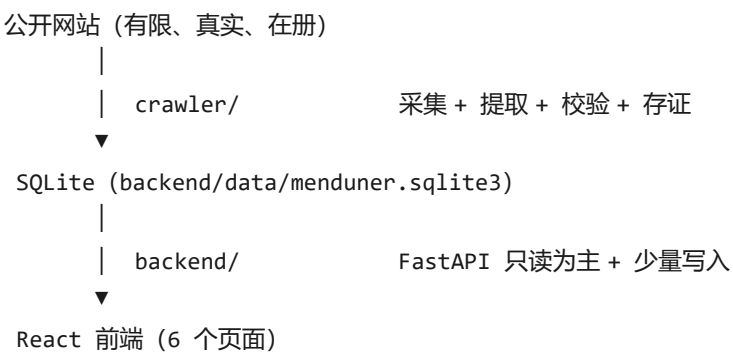
三条铁律

这三条是整个项目的地基，改动任何代码前都先确认没有违反：

- 不编造。缺失就写缺失，不确定就标待复核。宁可少一条活动，不许多一个假字段。
- 可追溯。任何一个进入系统的值，都必须能指回原始网页的出处。
- 不确定的交人工。自动核验只在官方来源 + 提取完全确定时才成立，其余全部进人工队列。

1 架构总览

四段式，单向流动，每一段都能独立测试：



关键点：采集和展示完全解耦。前端不直接碰数据库，也不能触发采集；采集失败不会让前端崩溃，只会让数据停止增长。

代码规模（实测）

目录	文件数	行数	职责
crawler/	68	11,678	采集、提取、校验、存证、核验规则、采集日志
frontend/src/	34	5,291	6 个页面 + 组件 + API 客户端
scripts/	19	2,809	一次性维护脚本（补坐标、去重、二维码等）
backend/	10	1,188	FastAPI 路由、数据投影、审核存储

模块地图

文件	职责	改它的风险
crawler/source_registry.yaml	所有来源的注册表：URL、信任等级、适配器、状态	低（加来源就改这里）
crawler/pipeline/__init__.py	发布管线：指纹、去重、写库、事务	高（涉及数据一致性）
crawler/verification.py	自动核验规则，全项目唯一的判定入口	极高（改了会污染核验状态）
crawler/extractor/	各来源的字段提取	中
crawler/validator/	提取结果校验，决定丢弃还是保留	高
crawler/review.py	人工审核决定的持久化存储	高
crawler/http.py	受限 HTTP 客户端：robots、限速、重试	中（影响所有来源）
crawler/venues.py	场馆身份归一（别名表）	中

backend/main.py	所有 API 路由 + 数据投影	中
backend/database.py	schema 初始化与连接	高（迁移）

2 数据模型

SQLite 单文件。表分成四类，理解分类就不会混：

① 业务实体（前端直接读）

表	行数	说明
events	134	活动。列名用前端驼峰：name / startDate / venueName …
venues	51	场馆。payload 是 JSON，坐标、地址、别名都在里面
hotels	15	酒店。含 tier（高端/中端/舒适）和判定依据
districts	29	商圈/街道。来自高德反查，属推断数据

② 证据链（这是本项目的核心）

表	行数	说明
evidence	1,376	字段级证据缓存，前端读这张。列名用前端驼峰
crawler_evidence	1,364	字段级证据原始表，列名用后端下划线
crawler_raw_items	1,357	原始网页留存。只追加，不删除（历史留证）
crawler_observations	145	活动 ←→ 原始页面的连接表
crawler_unified_events	134	指纹表，跨来源去重用

注意 evidence 和 crawler_evidence 靠同一个 id 关联，但列名风格不同。写后端代码时用下划线，写前端时用驼峰，混用会静默拿不到值 —— 这个坑踩过一次，见 11 节。

③ 人工决定（只增不改）

表	行数	说明
review_decisions	97	每条人工审核决定：event_id + field + decision + reviewer
venue_edits	2	场馆坐标/合并的人工记录

④ 运行日志

表	说明

crawler_jobs	每次采集任务的完整报告 JSON
crawler_attempts	每个来源每次尝试的结果
crawl_runs	每来源每次运行：起止时间、状态、发现数、失败原因
crawl_alerts	连续失败 3 次自动记一条告警，恢复后自动消除
client_errors	前端崩溃上报，含堆栈和屏幕尺寸（上限 200 条自动清理）
runs / sources	旧表，保留兼容

字段状态四态

状态	含义	数量
Verified	已核验。官方来源且提取完全确定，或人工通过	335
Needs Review	待复核。进入人工队列	612
Missing	来源确实没有这个值（不是留空）	294
Unverified	有值但无法核验	135

Missing 和留空的区别很重要：留空看起来像没有这个字段，Missing 才说明是来源本来就没给。这个区别是人工审核能工作的前提。

3 采集管线

一次采集的完整流程，按顺序：

```
crawler/run.py                  CLI 入口，解析参数
      │
      ▼
crawler/registry.py             读 source_registry.yaml
      │
      ▼
crawler/http.py                 PublicClient（robots + 限速 + 重试）
      │
      ▼
crawler/sources/*.py            每个来源一个适配器，collect() 返回原始条目
      │
      ▼
crawler/extractor/              从原始 HTML 提取结构化字段
      │
      ▼
crawler/validator/              校验：日期是否明确、场馆是否可定位
      │
      ▼
crawler/pipeline/__init__.py    指纹 → 跨来源去重 → 事务写入
      │
      ▼
crawler/verification.py         对每个字段判定核验状态
      │
      ▼
crawler/crawl_log.py            记录本次运行结果，必要时告警
```

指纹与去重

同一个活动常出现在多个来源，标题标点不同（「——」vs「:」）。去重靠指纹：

```
fingerprint = hash([
    identity_text(name),        # 归一化：NFKC + 反转义 + 转小写 + 去标点空白
    start_date,
    identity_text(place),
])

# 归一化后任何一部分为空 → 不生成指纹，宁可不合并也不错合并
```

identity_text() 只做归一化，不做模糊匹配。相似度算法会把「格调室内乐·二」和「格调室内乐·三」合并，那是错的。

批量原子性

一批活动要么全部写入，要么全部不写。写入过程中抛异常会调用 withdraw_new_events() 回滚，避免库里留下半批数据。

4 新增来源的完整步骤

这是最常做的改动。按顺序做，不要跳步。

第 1 步：确认这个来源值得接

检查	为什么
robots.txt 允许抓取吗	不允许就不接，别绕。大麦就是这么被排除的
页面上有真实的未来活动吗	政府站常是「圆满落幕」式回顾，接了只会污染数据
是 HTML 静态渲染还是 JS 空壳	JS 空壳抓不到内容，苏州文化艺术中心就是这样
有明确的场馆和日期吗	只有日期没场馆的活动无法上图，价值有限

第 2 步：在注册表登记

编辑 crawler/source_registry.yaml，加一条：

```
- source_id: venue-example-hall
  source_name: 示例场馆
  category: venue                          # government/venue/event_platform/media/tourism
  url: https://example.com/
  crawl_url: https://example.com/events  # 实际抓的列表页
  trust_level: OFFICIAL_VENUE            # 决定能否自动核验，见第 5 节
  priority: 1
  tier: 1
  adapter: example_hall                  # 必须和代码里的适配器 key 一致
  implementation: event_extraction       # 或 audit_only（只审计不提取）
  require_events: true                   # 抓不到活动就标记失败
```

第 3 步：写适配器

在 crawler/sources/ 下新建，然后在 SOURCES 字典注册。最小结构：

```
class ExampleHallAdapter:
    source_id = "venue-example-hall"      # 必须和注册表一致，否则启动即报错

    def __init__(self, client, max_items=10):
        self.client = client
        self.max_items = max_items
        self.pages_collected = 0
        self.errors = []

    def collect(self):
        # 返回原始条目列表。异常要记录到 self.errors，不要吞掉
        ...
```

适配器只负责「拿到原始内容」，不要在里面做字段提取 —— 提取归 extractor，校验归 validator。混在一起以后没法单测。

第 4 步：写测试

至少两个：一个正常页面能提取出活动，一个残缺页面不会编造字段。参考 crawler/tests/test_visitsz.py 的写法。

第 5 步：跑一次并检查

```
# 只跑这个来源，限额很小，用独立数据库避免污染
.\backend\.venv\Scripts\python.exe -m crawler.run ^
    --source example_hall --max-items 2 ^
    --db %TEMP%\probe.sqlite3
```

看输出里的 crawlLog 和 attempts：status 是 Success / Partial / Failed，失败会带 reason。确认没问题再跑全量。

退出码约定

退出码	含义	systemd 怎么处理
0	全部来源成功	正常
2	跑完了，但有来源失败	通过 SuccessExitStatus=0 2 视为正常。部分来源永久不可采（robots 禁止、JS 空壳），不该让每天的任务都报失败
1	崩溃	算失败，会告警

5 核验规则（改动前必读）

这是全项目最需要小心的地方。规则只有一条，条件很窄：

```
自动标记 Verified  ⇔  (来源信任等级 ∈ {OFFICIAL_GOVERNMENT, OFFICIAL_VENUE})
                      AND
                      (提取方式是确定性的)

其余一切 → 进人工审核队列
```

确定性提取怎么判定

信号	说明
方法名前缀	explicit_ 或 official_ 开头才算确定性
启发式标记	方法名含 rule / heuristic / guess / inferred / fallback / default → 直接判为不确定（这个检查最先执行）
场馆绑定	场馆名已核验，且 id 是从那个名字推导出来的，才允许绑定

判定来源的四种 origin

origin	含义
auto:official_deterministic	自动核验通过
auto:verified_venue_binding	通过已核验的场馆名绑定
pending:aggregator_source	聚合平台来的，必须人工
pending:heuristic_extraction	启发式提取的，必须人工
pending:no_deterministic_evidence	没有确定性证据

两条容易搞错的事

- 来源可信 ≠ 字段可信。官方政府站的页面里也可能有一条「日期未知」的活动，来源信任等级高不代表这一条就能自动核验。
- 活动级状态忽略派生字段。venueId / districtId / category / sourceUrl / city 这些是推导出来的，不参与「整个活动是否已核验」的判定，否则永远达不到已核验。

人工决定的生命周期

决定	含义

APPROVE → Verified	审核人认可这个值
REJECT → Rejected	审核人认为值是错的
CORRECT → Corrected	审核人给出修正值
ABSENT → AbsentConfirmed	审核人确认来源确实没有这个字段

两条关键行为：

- 决定按 event_id + field + reviewed_value 存储。重新采集不会覆盖人工判断。
- 但来源的值变了，人工决定自动失效。ABSENT 只在字段仍然为空时才算数 —— 不能让审核人为一个他从没见过的值背书。

6 前端结构

页面	路由	作用
CityEventMap	/map	地图，类别分色 + 热点场馆
EventCalendar	/calendar	月视图活动密度
KnowledgeGraph	/graph	活动→场馆→商圈→酒店 关系图
HotelIntelligence	/hotels	15 家酒店分层 + 周边活动
DistrictInsights	/insights	商圈热度与酒店供给对比
EvidenceQuality	/evidence	逐字段人工核验 + 来源状态 + 前端错误

数据流

```
usePlatform()              全局状态：dataset / filters / dataMode
      │
      ├─ filterEvents()      按年/月/商圈/类别/归档过滤
      │
   └─ 页面          不自己 fetch，一律从 context 取
```

新增页面时不要自己调 API，加到 usePlatform 的 dataset 里，这样筛选和刷新会自动生效。

几个必须知道的约定

约定	原因
字段名用驼峰	前端类型定义是驼峰，后端 API 也投影成驼峰
antd 两字中文按钮有空格	「通过」渲染成「通 过」，测试要用正则 /通\s*过/ 匹配
表格查询用 queryAllByText	antd Table 会渲染隐藏的测量行，按单数查询会失败
一个测试文件只挂载一次	antd 的 responsiveObserver 是模块级的，cleanup() 不会注销它，同文件第二次挂载会在 React commit 阶段失败
账号密码输入框文本 ≥ 16px	否则 iOS Safari 会自动放大页面
页面各自套 ErrorBoundary	没有边界时任何异常会让整个 React 树卸载，变成白屏

地图

- 高德地图只在初始化时读一次容器尺寸。改容器高度后必须调用 map.resize()，否则表现为灰色条带或地图形状错误。Map/index.tsx 里已挂 ResizeObserver。
- 小屏不渲染 MiniMap —— 它是 200×150 的绝对定位浮层，会盖住三分之一手机屏幕。

7 API 契约

FastAPI 自动生成文档：本地 http://127.0.0.1:8000/docs，线上 http://47.116.133.230/api/docs（需要绕过认证时用 GET）。

读接口（不需要认证）

方法	路径	说明
GET	/dataset	一次性返回 events/venues/hotels/districts/evidence/status —— 前端只调这个
GET	/events	活动列表（后端列名）
GET	/events/{id}	单个活动
GET	/venues	场馆列表
GET	/evidence	证据列表
GET	/hotels	酒店列表
GET	/districts	商圈列表
GET	/status	数据统计与来源数
GET	/review	所有人工决定
GET	/review/{event_id}	某活动的决定
GET	/crawl/health	每来源最近运行状态与告警
GET	/crawl/runs	采集日志
GET	/client-errors	前端错误（隐私相关，也可开认证）
GET	/venues/duplicates	疑似重复场馆候选
GET	/venues/coordinates/needed	缺坐标的场馆
GET	/venues/edits	场馆人工修改记录

写接口（需要口令）

方法	路径	说明
POST	/review	单字段决定
POST	/review/event	一次核完一个活动的全部事实字段
DELETE	/review/{event_id}/{field}	撤回人工决定
POST	/venues/coordinates	设置场馆坐标（可只给坐标，后端自动解析出处）
POST	/venues/merge	合并重复场馆
POST	/client-error	前端错误上报（特例：不需要认证）

认证的边界（设计决定）

看数据不需要密码，改数据需要。理由：

- 如果看数据也要登录，密码输错会让整个网站看起来像坏了 —— 这个误解真实发生过。
- POST /client-error 被排除在外：浏览器自动上报不带凭据，要求认证会让错误上报一条都写不进来 —— 这个也真实发生过。

8 部署与运行

线上环境

项	值
服务器	阿里云轻量应用服务器，Ubuntu 24.04，2 vCPU / 2 GiB
公网 IP	47.116.133.230
代码目录	/srv/menduner（属主 menduner:menduner）
前端静态目录	/srv/menduner/frontend-dist
数据库	/srv/menduner/data/menduner.sqlite3
后端服务	systemd: menduner-backend
定时采集	systemd: menduner-crawl.timer，每天 08:00
AI 交叉核对	systemd: menduner-ai-crosscheck.timer，每天 08:30

部署前端必须遵守的两条

这两条是用真实故障换来的：

- 绝不删除旧资源。Vite 文件名带哈希，新旧不会冲突，而还持有旧 index.html 的浏览器会来请求旧文件名。删了就是 404。
- index.html 必须不缓存。哈希资源可以缓存一年，前提是 HTML 每次重新校验 —— 这是缓存失效契约。

用 deploy/deploy-frontend.sh 部署，它会自动从 nginx 配置读出真实目录，并在结束后用 curl 校验「服务器实际返回的 HTML 引用的文件是否都存在」。

本地开发

```
# 环境（首次）
cd backend && python -m venv .venv && .\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
cd ..\frontend && npm install

# 日常启动
.\backend\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
cd frontend && npm run dev

# 一键（Windows）
start.bat      # 启动后端+前端并打开浏览器
stop.bat       # 停止
```

后端必须从仓库根目录启动，不是 backend 目录内 —— 否则会报 No module named backend。

环境变量

变量	用途	必需
MENDUNER_DB_PATH	数据库路径，默认 backend/data/menduner.sqlite3	否
MENDUNER_VENUE_ALIASES	场馆别名表路径	否
AMAP_WEBSERVICE_KEY	高德 Web 服务 key，补坐标用	补坐标时
DEEPSEEK_API_KEY	AI 交叉核对用	跑 AI 核对时
VITE_AMAP_KEY	前端地图 Web 端 key	地图必需
AMAP_SECURITY_CODE	高德安全密钥，只在服务端/代理用	地图必需

9 测试与质量

当前状态：后端 445 个测试，前端 66 个测试，全部通过。

```
# 后端全部
.\backend\.venv\Scripts\python.exe -m pytest crawler/tests backend/tests -q

# 前端类型检查 + 测试
cd frontend
npx tsc -b
npx vitest run
```

测试要测什么

层	该测什么	不该测什么
适配器	正常页面能提取、残缺页面不编造、robots 被遵守	真实网络（用 fixtures）
核验规则	官方+确定性 → Verified；其余 → pending	具体来源的 HTML 结构
管线	去重正确、批量原子性、异常回滚	数据库实现细节
API	字段名投影正确、状态码	前端渲染
前端	用户可见行为（按钮禁用、错误提示）	内部 state 形状

一个真实教训：验证必须用外部基准

二维码生成器写了三次才对。最初的自检测试全部通过，但二维码扫不出来 —— 因为测试只验证「我写的编码器和我写的检查器一致」，那个循环永远自洽。真正抓到 bug 的是引入外部基准：用 OpenCV 解码、用 segno 逐位对比。

写测试时问自己：这个断言依赖我自己的实现吗？如果依赖，它证明不了正确性。

10 改动规范

场景	必须做
加来源	注册表 + 适配器 + 测试 + 小限额试跑
改核验规则	先说明为什么现有规则不够，再改，并补测试证明新规则不会放宽错误的情况
改数据库 schema	写迁移，不要直接改 initialize()（线上有真实数据）
加 API 字段	同时改后端投影和前端类型定义，否则字段在前端拿不到

改前端字段名	前端用驼峰、后端用下划线，改一处必须两处都对
修复前端崩溃	加 ErrorBoundary 或前端错误上报，保证下次能看到原因

提交前检查

```
.\backend\.venv\Scripts\python.exe -m pytest crawler/tests backend/tests -q
cd frontend && npx tsc -b && npx vitest run
cd .. && git status --short        # 确认没有提交数据库、密钥、虚拟环境
```

不要提交：backend/.venv、frontend/node_modules、SQLite 数据库、.env 及其变体、构建产物。

11 已知局限与陷阱

当前局限（系统里可见，不是隐藏问题）

局限	原因
4 个来源采集失败	SIPAC 无可提取事件；苏州文化艺术中心是 JS 空壳；大麦搜索页 robots 禁止；Subao 抽不出有效日期
18 个场馆没有坐标	小剧场、茶馆、livehouse，高德没有对应条目
活动级已核验为 0	需全部事实字段核验通过，人工审核尚未完成（字段级已核验 335 条）
商圈是推断数据	来自高德反查，不是来源给出的原始字段
不做收益预测、不对接 RMS	项目边界

踩过的坑（改代码前扫一眼）

现象	真实原因
核验点了没反应	人工决定用后端列名存储，而 API 和 UI 用前端列名，所有决定都静默失效
页面全白	没有 ErrorBoundary，任何异常卸载整棵 React 树
Unable to preload CSS	index.html 被缓存，引用的是上一次构建的哈希文件名
部署显示成功但网站没变	脚本默认目录和 nginx 实际目录不一致；校验只查了源目录，没查实际返回
错误上报一条都没记录	POST 上报被认证拦住（浏览器 fetch 不带凭据）
地图灰色条带	高德只在初始化读一次容器尺寸，改尺寸没调 resize()
测试静默超时无报错	jsdom 缺 MutationObserver，Testing Library 的查询抛在未处理拒绝里
同文件第二个测试挂载失败	antd responsiveObserver 模块级且不随 cleanup 注销

环境注意事项

- Python 用 backend/.venv，不要用根目录的 .venv（历史遗留）。
- Windows 控制台默认 GBK，跑 Python 一律加 -X utf8，否则中文输出乱码。
- PowerShell 的 > 重定向写 UTF-16，需要文本文件时用 Python 写。
- 提交脚本文件时注意编码：.bat 要 UTF-8 无 BOM，.ps1 要 UTF-8 带 BOM。

附录 A 常用脚本

脚本	用途
scripts/resolve_venues_by_address.py	用地址+名称在高德解析场馆坐标（dry-run 默认）
scripts/dedupe_events.py	合并指纹相同的历史重复活动（dry-run 默认）
scripts/enrich_venue_coordinates.py	补场馆坐标
scripts/enrich_hotels.py	补酒店分层与坐标
scripts/assign_districts.py	按坐标分配商圈
scripts/ai_enrich.py	AI 读原文交叉核对（只读，不写库）
scripts/confirm_venue_references.py	批量确认场馆引用
scripts/qr_code.py	生成系统二维码（SVG）
scripts/build_deck.py	生成汇报 PPT
scripts/build_doc.py	生成本文稿

附录 B 关键常量位置

常量	文件	值
AUTHORITATIVE_TRUST_LEVELS	crawler/verification.py	OFFICIAL_GOVERNMENT, OFFICIAL_VENUE
DETERMINISTIC_PREFIXES	crawler/verification.py	explicit_, official_
HEURISTIC_MARKER	crawler/verification.py	rule|heuristic|guess|inferred|fallback|default
ALERT_THRESHOLD	crawler/crawl_log.py	3 次连续失败
HOTSPOT_MIN	frontend/src/utils/data.ts	3 场活动算热点场馆
GRACE_DAYS	frontend/src/hooks/usePlatform.tsx	30 天归档宽限期

本文稿中所有数字均从运行中的系统读取，模块清单、数据表结构、核验常量与 API 路由均取自当前代码，未凭记忆书写。